

Assignment - 13

- 1) What is mean by Reference in C++? Explain with example.
→ Refer Assignment 12 - Q1
- 2) What is mean by Constructor & Destructor? When are they called.
→ Refer Assignment 11 - Q3
- 3) Write down the rules which must be followed while writing Constructor & Destructor.
→ Refer Assignment 11 - Q3
- 4) What are different types of Constructors? Explain with example sample program.

Types of
Constructors in
C++

→ Default
→ Parameterized
→ Copy

- 1) Default - when we create an object w/o passing any parameter then default constructor gets called.
- 2) Parameterized - when we create an object by passing required parameters then parameterized constructor gets called.
- 3) Copy - when we create an constructor by copying an object then the copy constructor.

```
#include <iostream>
```

```
using namespace std;
```

```
Class Demo
```

```
{ public :
```

```
    int i;
```

```
    int j;
```

```
Demo ()
```

```
{ cout << " Default constructor \n"; }
```

```
Demo (int A, int B)
```

```
{ i = A;
```

```
  j = B;
```

```
  cout << " Parameterized Constructor \n ";
```

```
}
```

```
Demo (Demo &ref)
```

```
{ i = ref.i;
```

```
  j = ref.j;
```

```
  cout << " Copy constructor \n ";
```

```
}
```

```
~ Demo ()
```

```
{ cout << " Destructor \n"; }
```

```
int main()
```

```
{ Demo obj1;
```

```
  Demo obj2 (11, 12);
```

```
  Demo obj3 (obj2); }
```


Q.5] What is meant by this pointer? What is the prototype of this pointer? Explain with example.

- 1) In C++, to call any non-static function/method we need an object of that class.
- 2) By using object & direct accessing operator (.), we can call any non-static method.
- 3) When we use that caller object internally the compiler will sense the address of that object as 'first hidden implicit parameter'.
- 4) That address gets stored in '1st implicit argument' of that function & that argument is called as This Pointer.
- Keyword (C++ & Java) → 'this'

```
#include <iostream>
```

```
using namespace std;
```

```
Class Demo {  
    public:
```

```
    int i, j, k;
```

```
    Demo ( int a = 10, int b = 20, int c = 30 )
```

```
    { this → i = a;
```

```
      this → j = b;
```

```
      this → k = c;
```

```
    void display ()
```

```
    { cout << "Object Address: " << this << endl;
```

```
      cout << "Value - i: " << i << ", j: " << j << ", k: " << k << endl;
```

```
    }
```

```
};
```

```
int main ()
```

```
{ Demo obj (10, 50, 60);
```

```
  obj.display ();
```

```
  return;
```

```
}
```

Output

Object Address: 0x7fff78207820

Value -

i: 10, j: 50, k: 60

Q7] What is Inheritance ? Explain types of Inheritance with program.

-
- One of the object oriented Paradigm (C++ & Java)
 - Reusability
 - One class can acquire char & beh of other class

Types of Inheritance

- Single Level
- Multi Level
- Multiple Level

Q1] Single Level (^{Demo}Parent → ^{Hello}child)

```
#include <iostream>
```

```
using namespace std;
```

```
Class Demo {
```

```
// Demo Class
```

```
public:
```

```
    int x, y;
```

```
    Demo() {
```

```
        cout << "Demo Constructor" << endl;
```

```
        x = 10;
```

```
        y = 20;
```

```
    }
```

```
    ~ Demo() {
```

```
        cout << "Demo Class" << endl;
```

```
        x = 10;
```

```
        y = 20;
```

```
    }
```

```
    void fun() {
```

```
        cout << "Inside fun of Demo class" << endl;
```

```
    };
```



```
class Hello : public Demo    // Hello = Derived
{
public:
```

```
    int a, b, c;
```

```
    Hello ()
```

```
    {
        cout << "2. Hello constructor" << endl;
```

```
        a = 30;
```

```
        b = 40;
```

```
        c = 50;
```

```
    }
```

```
    ~Hello ()
```

```
    {
        cout << "2. Hello Destructor" << endl;
```

```
        void gun ()
```

```
        {
            cout << "Inside gun of Hello" << endl;
        }
```

```
    }
```

```
};
```

```
int main ()
```

```
{
    Hello hobj;
```

```
    cout << sizeof(hobj) << " bytes" << endl;
```

```
    cout << hobj.x << endl;
```

```
    cout << hobj.y << endl;
```

```
    cout << hobj.a << endl;
```

```
    cout << hobj.b << endl;
```

```
    hobj.fun();
```

```
    hobj.gun();
```

```
    return 0;
```

```
}
```

* Here all props & beh of Demo class are inherited by Hello class.

2] Multi Level (Grandfather → father → child)

- Extension of single level inheritance.

- Atleast 3 classes

#include <iostream>

using namespace std;

class Demo {

public:

int x, y;

Demo()

{ cout << "1. Demo constructor" << endl;

x = 10;

y = 20;

}

~Demo()

{ cout << "Demo Destructor" << endl;

x = 10;

y = 20;

}

void fun()

{ cout << "Inside fun of Demo class" << endl; }

}

class Hello : public Demo

{ public:

int a, b, c;

Hello()

{ cout << "2. Hello constructor" << endl;

a = 30;

b = 40;

c = 50;

}

~Hello()

{ cout << "2. Hello Destructor" << endl; }

void gun()

{ cout << "inside gun of Hello" << endl; }

}


```
class jsm : public Hello
{ public:
```

```
    int p;
```

```
    jsm()
```

```
{ cout << "3. jsm constructor" << endl;
```

```
    p = 60;
```

```
}
```

```
~ jsm()
```

```
{ cout << "3. jsm destructor" << endl; }
```

```
void sun()
```

```
{ cout << "Inside sun of jsm" << endl; }
```

```
int main()
```

```
{ jsm jsmob;
```

```
  Hello hob;
```

```
  cout << sizeof(jsmob) << " bytes" << endl;
```

```
  cout << jsmob.x << endl;
```

```
  cout << jsmob.y << endl;
```

```
  _____ .a << endl;
```

```
  _____ .b << endl;
```

```
  _____ .c << endl;
```

```
  _____ .p << endl;
```

```
  jsmob.func()
```

```
  jsmob.gunc()
```

```
  jsmob.sun()
```

```
  return 0;
```

```
}
```

3) Multiple

— One can inherit ≥ 1 classes at a time.

include <iostream>
using namespace std;

class Demo {

public:

int x, y;

Demo()

{ cout << "1. Demo Constructor" << endl;

x = 10;

y = 20;

}

~ Demo()

{ cout << "2. Demo Destructor" << endl;

x = 10;

y = 10;

}

void fun()

{ cout << "Inside fun of Demo" << endl; }

};

class Hello

{ public:

int a, b, c;

Hello()

{ cout << "2. Hello Constructor" << endl; }

a = 30;

b = 40;

c = 50;

}

~ Hello()

{ cout << "2. Hello Destructor" << endl; }

void gun()

{ cout << "Inside gun of Hello" << endl; }

Class Jsm: public Demo, public Hello
{ public:

int p

Jsm()

{ cout << "Jsm Constructor" << endl;

p = 60;

}

~Jsm()

{ cout << "Jsm Destructor" << endl; }

void sun()

{ cout << "Inside sun of Jsm" << endl; }

};

int main()

{ Jsm jsmob;

Hello obj;

Cout << jsmob.x << endl;

_____ . y _____;

_____ . a _____;

_____ . b _____;

_____ . c _____;

_____ . p _____;

jsmob.

jsmob.

jsmob.

return 0;

}

#1 What is meant by Polymorphism? Explain its types with program.

- Single name & multiple behaviours

Types $\left\{ \begin{array}{l} \text{Compile time (early binding)} \rightarrow \text{overloading} \\ \text{Runtime (late time binding)} \rightarrow \text{overriding} \end{array} \right.$

1) Concept of Compile time polymorphism is considered as Early binding, becoz the decision taken by compiler is in early phase.

To achieve this in C++, we have to use the term called overloading $\left\{ \begin{array}{l} \text{a) Function overloading} \\ \text{b) Operator overloading} \end{array} \right.$

2) Runtime - occurs when a function call is resolved at runtime. It is achieved through $\rightarrow$ overriding.

~~#~~ Include <iostream>

using namespace std;

Class CompileTime {

public:

void display (int x)

{ cout << "Integer: " << x << endl; }

void display (double y)

{ cout << "Double: " << y << endl; }

};

Class Base:

{ public:

void show () override

{ cout << "Derived Class" << endl; }

};

int main()

{ Compile Time obj ;

obj.display (10);

obj.display (3.14);

Base * Baseptr;

Derived d;

Baseptr = &d;

Baseptr → show();

return 0;

}

output

Integer : 10

Double : 3.14

Derived class

Q7 What is the use of & operator in case of - Copy Constructor
→ The & operator used inside the copy constructor is considered as the 'reference' operator. If this operator is not written then it may lead to recursive calls.

Ex. Class MyClass {

public:

int value;

MyClass (const MyClass &obj) {

value = obj.value;

}

};

Q7 Why the name of Constructor & Destructor is same as class name?

→ They have same name as the class to ensure they are easily identifiable & associated with class.
→ This allows the compiler to automatically call them during object creation & destruction without explicit invocation, ensuring proper initialization & cleanup resources.

10/ What is meant by Overloading? Explain function overloading.

→ Overloading refers ability to define multiple functions or operators with same name but different parameters. According to concept of function overloading, all the functions are having same name but different prototype.

It takes place only in a single class

Ex. `int add (int a, int b);`

`float add (float a, float b);`

- According to J.D. Rorer, polymorphism is considered as syntactic sugar, which adds flavour in any application.